Universidade Estadual de Londrina

Departamento de Computação

1COP009 - Programação Orientada a Objeto **Prof. Bruno B. Zarpelão**

Avaliação 2

09/12/2021

Nome do aluno: _____

1. (1,0) Observe as classes a seguir:

```
public class Veiculo {
  private String renavam;
  private String placa;
  public Veiculo(String renavam, String placa) {
    this.renavam = renavam;
    this.placa = placa;
  }
}

public class Caminhao extends Veiculo {
  private double capacidadeCarroceria;
  public Caminhao(String renavam, String placa,
                          double capacidadeCarroceria) {
    this.renavam = renavam;
    this.placa = placa;
    this.capacidadeCarroceria = capacidadeCarroceria;
  }
}
```

Quando tentamos compilá-las, os seguintes erros são apresentados:

- Implicit super constructor `Veiculo()` is undefined. Must explicitly invoke another constructor.
- The field `Veiculo.placa` is not visible.
- The field `Veiculo.renavam` is not visible.

Explique cada um desses erros e mostre a solução para eles.

2. (1,5) Observe as duas classes a seguir e escreva a classe `ContaBancaria` necessária para completar o código desse programa.:

```
public class ContaCorrente extends ContaBancaria {
  public double limite;
  public void sacar(double saque) {
    if (this.saldo + limite >= saque) {
      this.saldo = this.saldo-saque;
    }
```

```
    }
8 }

10 public class Principal {
     public static void main(String[] args) {
12     ContaBancaria c = new ContaCorrente();
       c.saldo = 1000;
14     ((ContaCorrente)c).limite = 500;
       c.depositar(1000);
16     c.sacar(800);
       System.out.println(c.saldo);
18   }
   }
```

3. (1,5) Observe a classe `Principal` apresentada a seguir. Ela foi implementada usando as classes `ContaBancaria` e `ContaCorrente` da questão anterior. A classe `Principal` tem, entre as linhas 7 e 12, seis diferentes atribuições, todas elas identificadas por um comentário. Para cada uma dessas atribuições, indique se há algum erro ou não. Se houver, aponte qual seria a solução.

```
1  public class Principal {
     public static void main(String[] args) {
3      ContaBancaria cb1;
       ContaBancaria cb2 = new ContaCorrente();
5      ContaCorrente cc1;
       ContaCorrente cc2 = new ContaCorrente();
7      cb1 = cb2;      //atrib1
       cc1 = cc2;      //atrib2
9      cb1 = cc2;      //atrib3
       cb1.saldo = 1000;   //atrib4
11     cb1.limite = 1000;  //atrib5
       cc2 = cb2;      //atrib6
13   }
   }
```

4. (1,5) Uma loja está vendendo árvores de Natal sob encomenda. Ela quer desenvolver um sistema que permita ao consumidor montar a sua própria árvore. A árvore sempre terá um componente básico, que é um pinheiro natural e poderá ser decorada com luzes, bolas de natal, festões, etc. A ideia do programa é imprimir na tela a descrição da árvore, detalhando os seus componentes. As classes a seguir são parte da implementação desse programa, que está seguindo o padrão de projetos Decorator. Seguindo as diretrizes desse padrão, escreva os códigos para as classes `ArvoreNatalDecorator`, `ArvoreNatalLuzes` e `ArvoreNatalFestoes` para completar o programa.

```
   public interface ArvoreNatal {
2    public String getDescricao();
```

```
}

public class ArvoreNatalBasica implements ArvoreNatal{
  public String getDescricao() {
    return " Pinheiro natural";
  }
}
```

5. (1,5) Observe o código a seguir. Ele implementa uma classe para gerenciar conexões com um banco de dados, utilizando as classes `DriverManager` e `Connection` que são disponibilizadas pela biblioteca do banco de dados. Modifique a classe `ConexaoBD` para que ela seja implementada de acordo com o padrão de projeto Singleton.

```
public class ConexaoBD {
  private Connection conexao;
  public ConexaoBD() {
    this.conexao = DriverManager.getConnection("192.168.0.1:2000");
  }
  public Connection getConexao() {
    return this.conexao;
  }
}
```

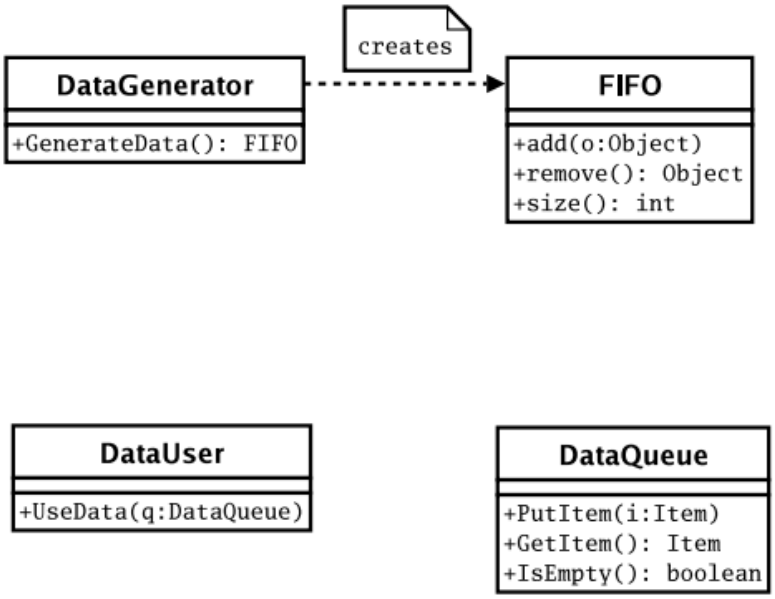
6. (1,5) Observe o código do programa a seguir. A ideia é termos um método que busca uma música MP3 em uma lista de acordo com diferentes critérios, como título da música ou duração. Reescreva o código para a classe `BuscadorMusicas` e escreva o código para nova(s) classe(s) que seja(m) necessária(s) para que esse programa seja implementado de acordo com o padrão de projeto Template.

```
public class MusicaMp3 {
  public String titulo;
  public String album;
  public String artista;
  public int duracaoSegundos;
  public MusicaMp3(String titulo, String album,
                   String artista, int duracaoSegundos) {
    super();
    this.titulo = titulo;
    this.album = album;
    this.artista = artista;
    this.duracaoSegundos = duracaoSegundos;
  }
}

import java.util.LinkedList;
import java.util.List;
```

```java
public class BuscadorMusicas {
  public static final int TITULO_IGUAL_A = 1;
  public static final int DURACAO_IGUAL_A = 2;
  public int buscarMusica(List<MusicaMp3> listaMusicas,
                  MusicaMp3 busca, int atributo) {
    int posicao = 0;
    for (MusicaMp3 musica:listaMusicas) {
      if (atributo == TITULO_IGUAL_A) {
        if (musica.titulo.equals(busca.titulo)) {
            return posicao;
        }
      } else if (atributo == DURACAO_IGUAL_A) {
        if (musica.duracaoSegundos ==
                            busca.duracaoSegundos) {
          return posicao;
        }
      }
      posicao++;
    }
    return -1;
  }
}
```

7. (1,5) Suponha que nós temos 4 classes em um sistema conforme vemos na figura a seguir:



A classe `DataGenerator` gera alguns dados e os coloca em uma fila, que é gerenciada por meio da classe `FIFO`. A classe `DataUser`, por sua vez, precisa consumir esses dados, mas foi implementada para fazer isso por meio da classe `DataQueue`. Escreva o código da classe Adapter que vai permitir que a classe `DataUser` acesse a classe `FIFO` como se ainda estivesse acessando a classe `DataQueue`.